Git Feature Branch Workflow – Complete Notes

Objective

Learn the complete Git workflow used in real software development projects.

By the end of this practice, you will understand how to:

- Create a GitHub repository
- Create feature branches
- Clone a repository to your local machine
- Switch between branches
- Modify files
- Commit changes
- Push changes to GitHub
- Create a Pull Request (PR)
- Merge into the main branch
- Keep your local repository updated

Why Do We Use Feature Branches?

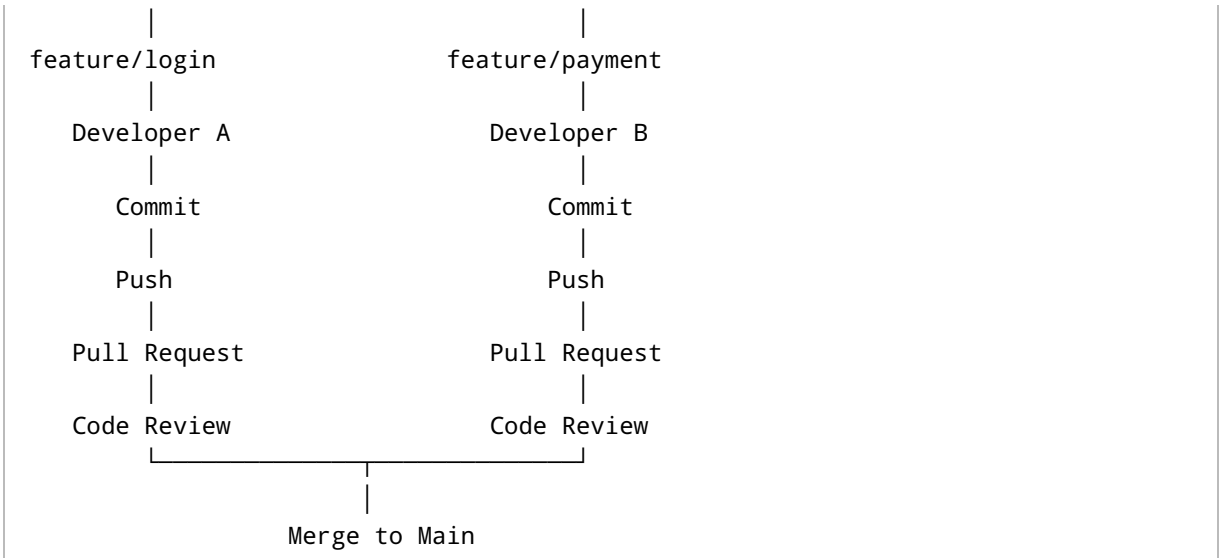
In real projects, developers never work directly on the **main** branch.

Instead, every new feature, bug fix, or enhancement is developed in a separate branch.

This provides several benefits:

- Protects the main branch from unstable code
- Allows multiple developers to work simultaneously
- Enables code reviews before merging
- Makes it easier to track changes
- Simplifies rollback if issues occur

Real-World Workflow



Repository Structure

Repository Name

```
Git-Practice
```

Repository contains

```
Git-Practice/

├── README.md
├── demo.txt
```

README.md

```
# Git Practice
```

demo.txt

```
This is my first Git practice.
```

Step 1 – Create GitHub Repository

1. Login to GitHub.
2. Click **New Repository**.
3. Repository Name:

```
Git-Practice
```

1. Select

2. Public (or Private)

3. Initialize with README

4. Click **Create Repository**.

Step 2 – Create Initial Files

Create

```
demo.txt
```

Content

```
This is my first Git practice.
```

Commit directly to

```
main
```

Repository now contains

```
README.md

demo.txt
```

Step 3 – Create Feature Branch

Instead of modifying **main**, create a feature branch.

Example

```
feature/update-demo
```

The repository now becomes

```
main

feature/update-demo
```

Why Not Work on Main?

Wrong approach

```
Developer

↓

main

↓

Commit

↓

Production
```

Correct approach

```
Developer

↓

Feature Branch
```

```
↓

Commit

↓

Push

↓

Pull Request

↓

Review

↓

Merge

↓

Main
```

Step 4 – Clone Repository

Clone repository to your local computer.

```
git clone https://github.com/<username>/Git-Practice.git
```

Move into repository

```
cd Git-Practice
```

Step 5 – Check Current Branch

```
git branch
```

Output

```
* main
```

Meaning

- `*` indicates the currently active branch.

Step 6 – Fetch Latest Branch Information

```
git fetch
```

Purpose

Downloads information about new branches and commits from GitHub without modifying your local files.

Step 7 – View All Branches

```
git branch -a
```

Example

```
* main

remotes/origin/main

remotes/origin/feature/update-demo
```

Explanation

Local branches

```
main
```

Remote branches

```
origin/main

origin/feature/update-demo
```

Step 8 – Switch to Feature Branch

If the branch already exists locally

```
git checkout feature/update-demo
```

If the branch exists only on GitHub

```
git checkout -b feature/update-demo origin/feature/update-demo
```

Verify

```
git branch
```

Output

```
main

* feature/update-demo
```

Step 9 – Modify Files

Open

```
demo.txt
```

Before

```
This is my first Git practice.
```

After

```
This is my first Git practice.

I am learning Git.

I created a feature branch.

I will merge this into main.
```

Save the file.

Step 10 – Check Changes

```
git status
```

Example

```
modified: demo.txt
```

Meaning

Git has detected changes but they are not yet staged.

Step 11 – Stage Changes

Stage a specific file

```
git add demo.txt
```

Stage all files

```
git add .
```

Step 12 – Commit Changes

```
git commit -m "Updated demo.txt with practice changes"
```

Purpose

Creates a snapshot of your staged changes in the local Git history.

Step 13 – Push Feature Branch

```
git push origin feature/update-demo
```

Purpose

Uploads your local commits to GitHub.

Step 14 – Create Pull Request

Go to GitHub.

Click

```
Compare & Pull Request
```

Base branch

```
main
```

Compare branch

```
feature/update-demo
```

Add

Title

```
Updated demo.txt
```

Description

```
Added practice changes to demo.txt.
```

Click

```
Create Pull Request
```

Step 15 – Code Review

Team Lead or Senior Developer reviews the code.

Possible outcomes

- Approved
- Changes Requested
- Comments Added

Only after approval is the Pull Request merged.

Step 16 – Merge Pull Request

Click

```
Merge Pull Request
```

Then

```
Confirm Merge
```

The feature branch is merged into the **main** branch.

Step 17 – Update Local Main Branch

Switch to main

```
git checkout main
```

Download the latest changes

```
git pull origin main
```

Your local main branch is now synchronized with GitHub.

Complete Command Flow

```
git clone https://github.com/<username>/Git-Practice.git

cd Git-Practice

git branch

git fetch

git branch -a

git checkout -b feature/update-demo origin/feature/update-demo

# Edit files

git status

git add .

git commit -m "Updated demo.txt"

git push origin feature/update-demo

# Create Pull Request on GitHub

# Merge Pull Request

git checkout main
```

```
git pull origin main
```

Common Git Commands

Command	Purpose
`git clone`	Copy repository from GitHub
`git status`	Check current changes
`git fetch`	Download latest branch information
`git branch`	Show local branches
`git branch -a`	Show local and remote branches
`git checkout <branch>`	Switch to an existing branch
`git checkout -b <branch>`	Create and switch to a new branch
`git add .`	Stage all changes
`git commit -m "message"`	Save staged changes locally
`git push origin <branch>`	Upload commits to GitHub
`git pull origin main`	Download latest changes from main
`git log --oneline`	View commit history
`git branch -d <branch>`	Delete a local branch
`git push origin --delete <branch>`	Delete a remote branch

Interview Questions

Why should we not work directly on the main branch?

- Prevent accidental issues in production
- Enable code reviews
- Allow multiple developers to work independently
- Maintain a stable main branch

What is a feature branch?

A feature branch is a separate branch created from the main branch to develop a specific feature or enhancement without affecting the stable codebase.

What is a Pull Request?

A Pull Request (PR) is a request to merge changes from one branch into another. It allows team members to review, discuss, and approve code before it becomes part of the main branch.

Difference Between `git fetch` and `git pull`

`git fetch`

- Downloads the latest commits and branch information from the remote repository.
- Does **not** modify your current working branch.

`git pull`

- Performs a `git fetch`.
- Immediately merges the downloaded changes into your current branch.

Best Practices

- Never commit directly to the `main` branch.
- Create a separate feature branch for every task.
- Write meaningful commit messages.
- Pull the latest changes before starting new work.
- Push changes regularly.
- Always create a Pull Request.
- Review code before merging.
- Delete feature branches after they are merged to keep the repository clean.

Complete Development Workflow

```
Create Repository
        |
Create Feature Branch
        |
Clone Repository
```

```
        |
Fetch Latest Changes
        |
Switch to Feature Branch
        |
Modify Files
        |
git status
        |
git add
        |
git commit
        |
git push
        |
Create Pull Request
        |
Code Review
        |
Merge to Main
        |
git checkout main
        |
git pull origin main
        |
Repository Updated Successfully
```

Summary

This workflow represents the standard Git development process followed by most software companies. Every change is developed in a feature branch, reviewed through a Pull Request, and merged into the main branch only after approval. Following this process helps teams collaborate efficiently, maintain code quality, and keep the main branch stable.